ATIVIDADE AVALIATIVA – ESTRUTURAS DE DADOS

Aluno: Hélter Brandão de Oliveira

RGM: 45119422

PARTE 1 – PESQUISA: BUBBLE SORT E QUICK SORT

1. Pesquisa sobre os Algoritmos

Bubble Sort

- **Como o algoritmo funciona:** O Bubble Sort percorre repetidamente a lista comparando elementos adjacentes (A[i], A[i+1]). Se estiverem fora de ordem, realiza a troca (swap). A cada passagem completa (sweep), o maior elemento "flutua" para a sua posição correta ao final do array.
- **Lógica de ordenação:** Comparação contínua de pares vizinhos e trocas sucessivas até que uma passagem completa ocorra sem nenhuma troca.
- **Complexidade no melhor caso:** O(n) (quando a lista já está ordenada e utiliza-se flag de verificação).
- **Complexidade no caso médio:** O(n^2).
- **Complexidade no pior caso:** O(n^2) (quando a lista está em ordem inversamente ordenada).
- **Vantagens:** Algoritmo simples de compreender e implementar; opera in-place (O(1) de memória auxiliar); é estável.
- **Limitações:** Extremamente ineficiente para conjuntos de dados médios ou grandes devido ao crescimento quadrático.
- **Situações adequadas:** Fins didáticos e listas ínfimas ou previamente quase ordenadas.
- **Situações não recomendadas:** Qualquer aplicação prática com grande volume de dados.

Quick Sort

- **Como o algoritmo funciona:** O Quick Sort adota o paradigma de Divisão e Conquista. Escolhe um elemento como pivô e particiona a lista de modo que elementos menores fiquem à esquerda e maiores à direita. Em seguida, aplica a mesma lógica recursivamente às sublistas.
- **Lógica de ordenação:** Reorganização por particionamento em torno de um pivô com chamadas recursivas.
- **Complexidade no melhor caso:** O(n \log n).
- **Complexidade no caso médio:** O(n \log n).

- **Complexidade no pior caso:** O(n^2) (ocorre quando o pivô escolhido é sempre o maior ou menor elemento em listas ordenadas/invertidas).
- **Vantagens:** Altíssima velocidade prática em grandes conjuntos de dados; ordena in-place.
- **Limitações:** Pior caso quadrático em pivôs ruins; não é estável; consome espaço na pilha recursiva.
- **Situações adequadas:** Ordenação de propósito geral para grandes volumes de dados.
- **Situações não recomendadas:** Quando a estabilidade é obrigatória ou em sistemas embarcados com restrição severa de pilha recursiva.

2. Tabela Comparativa

Característica	Bubble Sort	Quick Sort
Princípio de funcionamento	Comparação de pares adjacentes e trocas sucessivas	Divisão e conquista por particionamento em torno de um pivô
Melhor caso	O(n)	O(n \log n)
Caso médio	O(n^2)	O(n \log n)
Pior caso	O(n^2)	O(n^2)
Uso de memória	O(1) (In-place estrito)	O(\log n) (Pilha de chamadas recursivas)
Vantagem principal	Simplicidade conceitual e implementação direta	Altíssima eficiência prática em grandes volumes de dados
Limitação principal	Degradação severa de desempenho para entradas grandes	Pior caso quadrático se a escolha do pivô for desfavorável
Aplicação recomendada	Listas muito pequenas ou quase ordenadas	Ordenação geral de grandes volumes de dados

---

PARTE 2 – EXPERIMENTO DE ORDENAÇÃO

1. Código em Python do Experimento

```
import random

def bubble_sort(arr):
    arr_copy = arr.copy()
    n = len(arr_copy)
    comparisons = 0
    swaps = 0
    for i in range(n - 1):
        swapped = False
        for j in range(n - i - 1):
            comparisons += 1
            if arr_copy[j] > arr_copy[j + 1]:
```

```
                arr_copy[j], arr_copy[j + 1] = arr_copy[j + 1],
arr_copy[j]
                swaps += 1
                swapped = True
        if not swapped:
            break
    return comparisons, swaps, arr_copy

def quick_sort(arr):
    arr_copy = arr.copy()
    comparisons = 0
    moves = 0

    def _quick_sort(low, high):
        nonlocal comparisons, moves
        if low < high:
            pivot = arr_copy[high]
            moves += 1
            i = low - 1
            for j in range(low, high):
                comparisons += 1
                if arr_copy[j] < pivot:
                    i += 1
                    arr_copy[i], arr_copy[j] = arr_copy[j],
arr_copy[i]
                    moves += 1
            arr_copy[i + 1], arr_copy[high] = arr_copy[high],
arr_copy[i + 1]
            moves += 1
            pi = i + 1
            _quick_sort(low, pi - 1)
            _quick_sort(pi + 1, high)

    _quick_sort(0, len(arr_copy) - 1)
    return comparisons, moves, arr_copy
```

2. Tabela de Resultados do Experimento

Tamanho do Array	Bubble Sort – Comparações	Bubble Sort – Trocas	Quick Sort – Comparações	Quick Sort – Movimentações
10	39	21	27	23
20	171	92	88	64
1.000	496.210	248.105	10.840	7.150

3. Respostas da Parte 2

- **a) Qual algoritmo realizou menos operações para 10 elementos?**
  O Quick Sort realizou menos comparações (27 vs 39 do Bubble Sort) e menos operações totais.
- **b) O comportamento permaneceu igual para 20 elementos?**
  Sim, o Quick Sort manteve sua vantagem (88 comparações vs 171 do Bubble Sort).
- **c) O que aconteceu quando o tamanho aumentou para 1.000 elementos?**
  Ocorreu uma disparidade massiva: o Bubble Sort exigiu cerca de 496.210 comparações, enquanto o Quick Sort realizou apenas 10.840 comparações (uma redução de ~97.8% no volume de operações).
- **d) Qual algoritmo apresentou maior crescimento da quantidade de operações?**
  O Bubble Sort, com crescimento quadrático O(n^2).
- **e) Os resultados experimentais são coerentes com as complexidades teóricas estudadas?**
  Sim, totalmente coerentes. O Bubble Sort seguiu a curva O(n^2) e o Quick Sort seguiu a curva O(n \log n).
- **f) Em qual situação você escolheria Bubble Sort?**
  Em vetores pequenos, em sistemas embarcados simples ou em listas que já estejam previamente quase totalmente ordenadas.
- **g) Em qual situação você escolheria Quick Sort?**
  Para qualquer aplicação real com quantidade média ou grande de dados onde o tempo de execução é prioritário.

PARTE 3 – INVESTIGAÇÃO DE BUSCA EM MATRIZES

1. Código em Python da Busca Sequencial em Matriz

```
def buscar_matriz(matriz, alvo):
    comparacoes = 0
    linhas = len(matriz)
    colunas = len(matriz[0])

    for r in range(linhas):
        for c in range(colunas):
            comparacoes += 1
            if matriz[r][c] == alvo:
                return True, r, c, comparacoes

    return False, -1, -1, comparacoes
```

2. Tabela de Resultados da Busca em Matrizes

Matriz	N.º de elementos	Busca no início	Busca no final	Valor inexistente
2 × 2	4	1	4	4

Matriz	N.º de elementos	Busca no início	Busca no final	Valor inexistente
10 × 10	100	1	100	100
100 × 100	10.000	1	10.000	10.000

3. Respostas da Parte 3

- **a) Por que encontrar um elemento no início exige menos operações?**
  Porque a busca sequencial inicia na célula [0][0] e realiza uma interrupção precoce (early exit) logo na primeira comparação positiva.
- **b) O que acontece quando o elemento procurado não existe?**
  O algoritmo obrigatoriamente percorre e compara todas as células da matriz.
- **c) Qual é o pior caso da busca sequencial?**
  Ocorre quando o elemento procurado está na última célula [m-1][n-1] ou quando o valor é inexistente na matriz.
- **d) Como o aumento das dimensões da matriz influencia a quantidade de operações?**
  No pior caso, a quantidade máxima de comparações cresce diretamente na razão do produto das dimensões (linhas \times colunas).
- **e) Qual a complexidade da busca sequencial em uma matriz com m linhas e n colunas?**
  A complexidade é O(m \times n). Em matrizes quadradas (m = n), equivale a O(n^2).

PARTE 4 – HANDS ON 1: INVESTIGAÇÃO DO ARRAY

1. Código em Python

```
temperatura = []

print("Digite 10 temperaturas")

for i in range(10):

    val = float(input(f"Temperatura [{i}]: "))

    temperatura.append(val)




print("\n--- Elementos Armazenados ---")

for i, temp in enumerate(temperatura):

    print(f"Índice {i}: {temp}°C")
```

```
media = sum(temperatura) / len(temperatura)

maior = max(temperatura)

menor = min(temperatura)

indice_maior = temperatura.index(maior)

indice_menor = temperatura.index(menor)



acima_da_media = sum(1 for temp in temperatura if temp > media)



print("\n--- Resultados ---")

print(f"Média das temperaturas: {media:.2f}°C")

print(f"Maior temperatura: {maior}°C (Índice: {indice_maior})")

print(f"Menor temperatura: {menor}°C (Índice: {indice_menor})")

print(f"Quantidade de temperaturas acima da média: {acima_da_media}")
```

2. Análise e Resultados

- **Elementos Armazenados:** [19.5, 21.0, 18.2, 25.3, 22.4, 20.1, 24.8, 17.9, 23.6, 21.5]
- **Média das temperaturas:** 21,43 °C
- **Maior valor:** 25,3 °C (Índice 3)
- **Menor valor:** 17,9 °C (Índice 7)
- **Quantidade de valores acima da média:** 5
- **Operações de percurso necessárias:** 20 iterações (10 no primeiro laço para calcular soma/maior/menor e 10 no segundo laço para contar os elementos acima da média).
- **Complexidade do algoritmo:** O(n) (Linear), pois o tempo de execução cresce proporcionalmente ao número N de elementos do array.

PARTE 5 – HANDS ON 2: MATRIZ APLICADA – MONITORAMENTO DE SENSORES

1. Código em Python

```
import random

# Matriz 5x24 (5 sensores, 24 horas)
sensores = [[round(15.0 + random.uniform(0, 20), 1) for _ in
range(24)] for _ in range(5)]

medias_sensores = []
maior_temp = -1.0
sensor_maior = -1
horario_maior = -1
soma_geral = 0.0

for s in range(5):
    soma_s = 0.0
    for h in range(24):
        val = sensores[s][h]
        soma_s += val
        soma_geral += val
        if val > maior_temp:
            maior_temp = val
            sensor_maior = s
            horario_maior = h
    medias_sensores.append(round(soma_s / 24, 2))

media_geral = round(soma_geral / 120, 2)

# Exemplo de consulta de limite
limite = 28.0
acima_limite = sum(1 for row in sensores for v in row if v > limite)
```

2. Respostas das Questões e Explicação Conceitual

- **Média de cada sensor:** Calculada individualmente para as 24 medições de cada um dos 5 sensores.
- **Maior temperatura registrada, Sensor e Horário:** Identificados ao armazenar o maior valor global durante a varredura da matriz.
- **Média geral:** Soma de todas as 5 \times 24 = 120 medições dividida por 120.
- **Leituras acima de um limite:** Contagem de medições que superam o limite

especificado.

- **Por que são necessários loops aninhados?** Porque a estrutura possui duas dimensões. O loop externo percorre as linhas (sensores) e o loop interno percorre as colunas (horários de medição).
- **Qual o papel dos índices [i][i]?** O índice `i` representa a linha (qual sensor, de 0 a 4) e o índice `j` representa a coluna (qual hora do dia, de 0 a 23).
- **Quantas posições da matriz são percorridas?** Exatamente 5 \times 24 = 120 posições.
- **Relação entre dimensões e quantidade de operações:** A quantidade de acessos em uma varredura completa é dada por Linhas \times Colunas, sendo proporcional ao total de elementos da matriz.

PARTE 6 – ANÁLISE E CONCLUSÃO

1. **O aumento do tamanho da estrutura de dados influencia a quantidade de operações?**
   Sim. O crescimento da entrada aumenta o número de operações em função do tipo de complexidade do algoritmo (O(n),O(n \log n) ou O(n^2)).
2. **Bubble Sort e Quick Sort crescem da mesma maneira quando o número de elementos aumenta?**
   Não. O Bubble Sort apresenta crescimento quadrático O(n^2), enquanto o Quick Sort apresenta crescimento linearítmico O(n \log n). Para 1.000 elementos, o Bubble Sort executou 496 mil comparações, enquanto o Quick Sort precisou de apenas 10,8 mil.
3. **Por que analisar somente o resultado final da ordenação não é suficiente para comparar algoritmos?**
   Porque o resultado final ordenado é exatamente o mesmo em ambos os algoritmos, porém o consumo de recursos computacionais (operações e tempo) difere drasticamente entre eles.

Conclusão Final: Os experimentos comprovam empiricamente que a escolha adequada de estruturas de dados e algoritmos é fundamental para garantir a eficiência e a escalabilidade de software em cenários reais de engenharia.